

AI Red Team Lab — Operations Guide

Architecture: OrbStack Era | Version: 1.0 | Date: 2026-09-19

Suggested home: /Volumes/Mackie/AI-Red-Team-Lab/reports/lab-operations-guide.md — commit it, this is the lab's manual.

1. Purpose

A self-contained environment for learning offensive AI/app security: a local LLM (attack target's brain), a vulnerable web app (Open WebUI — the attack target), and a Linux attack toolbox (Kali) — all sandboxed away from the operator's personal network. Everything turns on with a handful of commands and off just as cleanly. Nothing runs in the background uninvited.

Threat model: the lab must (a) be invisible to the operator's personal network, (b) contain anything executed inside the lab, (c) never auto-start or leak — every listener exists only because the operator started it.

2. Architecture

text

```
PERSONAL NETWORK (en0 — hotspot/Wi-Fi)          ← lab is
INVISIBLE from here
|
MacBook (host)
|— Ollama — bound to 127.0.0.1:11434 ONLY — models on
SSD
|— lab-up / lab-down          (the manual switch — no
daemons, no autostart)
|— OrbStack (Docker engine)
|   |
|   |— kali — attack toolbox (nmap, sqlmap, Burp's
partner...)
|   |— webui — Open WebUI v0.3.16 (THE TARGET)
|   |       published 127.0.0.1:3000 → 8080
|   (loopback only)
|       |
|       |— talks to Ollama via
host.orb.internal:11434
```

Key design decision: Ollama binds loopback only. Containers reach it through OrbStack's host proxy (host.orb.internal), which relays into loopback — so the model is reachable from the lab but from no network interface whatsoever. This is stronger than the old UTM-era bind (which used a lab subnet IP).

3. Component Inventory

COMPONENT	WHAT	LIVES AT	STATE ON COLD MAC
Ollama server	The model API (llama3.2:3b, mistral:7b, qwen3:8b)	started by lab-up from /usr/local/bin/ollama	✗ off
Models (5.2 GB+)	GGUF model files	<SSD>/Applications/ollama/models	on SSD
lab-up / lab-down / lab-check	The switch: start/stop/verify	~/bin/ (real scripts, not shell functions)	—
ol	Ollama CLI wrapper (points at loopback)	defined in ~/bin usage / alias	—
OrbStack	Docker engine	/Applications/OrbStack.app — must be open while using Docker	off until opened
kali container	Attack toolbox (nmap 7.99, sqlmap, hydra, nikto, whatweb, exploitdb, wordlists, netcat, python3)	image kali-lab, built from docker/kali/ Dockerfile	off

webui container	Open WebUI v0.3.16 (pinned) — the vulnerable target	image from ghcr.io	off
webui data	its DB (users, chats, uploads)	docker volume open-webui — survives compose down	persisted
Repo / configs	compose.yaml, Dockerfile, findings, intel	/Volumes/Mackie/AI-Red-Team-Lab/	on SSD
UTM + old Kali VM	RETIRED daily driver — kept for future genuinely-hostile experiments (real VM isolation)	/Volumes/Mackie/UTM/	off

Why v0.3.16: old enough to have a published CVE history (Phase-2 practice), new enough that release-note "fixes" are present — so the game is testing whether each fix is complete. Never use :latest — it moves under you and breaks reproducibility.

4. Network Map — who can reach what

ADDRESS	WHAT IT IS	OLLAMA THERE?	WEBUI THERE?
127.0.0.1 (Mac)	Mac loopback	✅ the only bind	✅ :3000 (published to loopback only)
host.orb.internal (from inside a container)	OrbStack's host relay	✅ (relays into loopback)	❌
webui (from inside a compose container)	container DNS	❌	✅ :8080
localhost inside a container	the container itself	❌	❌

en0 IP (Wi-Fi/ hotspot side)	personal network	✗ sealed	✗ sealed
LAN / internet	everyone else	✗	✗

The localhost trap (memorize): every container has its own private localhost. "It works on the Mac but empty in the container" is always an address problem, not a failure. From kali use webui:8080 / host.orb.internal:11434.

5. Security Posture — the guarantees

GUARANTEE	MECHANISM	VERIFY WITH
Model invisible to personal network	loopback-only bind	curl -m3 "http://\$(ipconfig getifaddr en0):11434/api/tags" → refused
Target invisible to personal network	loopback-only publish in compose	same idea, port 3000
No uninvited listeners	no LaunchAgent, no Login Items, Ollama GUI neutered (chmod -x), manual switch only	after reboot: lsof -nP -iTCP:11434 -sTCP:LISTEN → empty
Verified off-state	lab-down then lab-check	empty lsof = truly off, not icon-hidden
Resource caps (DoS fence)	OLLAMA_MAX_LOADED_MODELS=1, NUM_PARALLEL=1, KEEP_ALIVE=5m	in lab-up env
Personal network blind to lab	NAT segmentation (container traffic egresses as the Mac; no L2 adjacency to sniff) + loopback binds	nmap from outside finds nothing
Container escape-hatch honesty	OrbStack containers run at Mac trust level — not a hostile-code sandbox. For that, UTM VM remains available	—

The one rule that keeps all of this: the only thing that ever runs ollama

serve is lab-up. If a menu-bar Ollama icon ever reappears (e.g. after an app update), kill it: `pkill -f "/Applications/Ollama.app"`, then `lab-down && lab-up`.

6. ⚡ **START Ritual (cold Mac → attacking, ~3 min)**

text

```
0. Plug in the SSD (Mackie) – wait for Finder
1. lab-up                                → want: "models: /
Volumes/..." + 127.0.0.1:11434 LISTEN
2. cd /Volumes/Mackie/AI-Red-Team-Lab/docker
3. Open the OrbStack APP                  → Docker engine starts
(required for step 4)
4. docker compose up -d                  → want kali + webui Up
5. VERIFY (10 sec):
    curl -s http://localhost:3000/health                                     →
{"status":true}
    docker exec kali curl -s -m3 http://
host.orb.internal:11434/api/tags | head -c 60                               →

{"models":...
6. Seats:
    Browser → http://localhost:3000          (personas: admin /
test1 / test2)
    Burp → proxy on, Intercept OFF
    docker exec -it kali bash                (attack shell, when
needed)
```

Login order matters: first-ever signup = admin (config only). Personas: test1 = victim (owns files), test2 = attacker (steals). Admin never touches the IDOR tests — keeps verdicts clean.

7. 🌙 **SHUTDOWN Ritual**

text

```
exit                (leave kali shell)
docker compose down (target off – volume persists,
users/chats kept)
lab-down            (model off – verified: "ollama:
OFF")
lab-check           (empty = proof)
quit OrbStack
```

Nothing runs. Nothing listens. Unplug the SSD whenever.

8. 🔥 Active Mission Runbook — F1: Cross-User File Access (IDOR)

Claim under test: changelog says cross-user file leaks "fixed" in 0.3.16.
Verify or break it.

1. Victim (test1): upload secret.txt containing TOPSECRET-USERA-7391 in a chat → Burp history → POST /api/v1/files/ → copy "id"
2. Attacker (test2): log in → F12 → Application → Local Storage → copy token
3. Burp Repeater:http

```
GET /api/v1/files/<FILE_ID>/content HTTP/1.1
Host: localhost:3000
Authorization: Bearer <test2-token>
```

4. Also fire: GET /api/v1/files/<id> · GET /api/v1/files/<id>/data · GET /api/v1/files/ (listing = worse than read) · same set against the Knowledge-upload id
5. Verdict: 200 + marker = LEAKY (screenshot immediately) · 403 =

blocked · 404 = hidden · write findings/F1-file-idor.md, update intel row #1, commit

Intel queue after F1 (from release-notes recon): #2 signup-disable bypass via API · #3 call/temporary-chat URL params · #6 shared-chat visibility toggle vs API · #4/#5 Functions = admin Python exec + auto-pip (the RCE endgame, needs a privilege chain first) · #8 Ollama proxy in-app (app-bug → model-management chain).

9. Troubleshooting — every gremlin this lab has produced, and the fix

SYMPTOM	CAUSE	FIX
lab-up: "SSD not found"	SSD unplugged, or volume mounted under a different name (OrbStack share vs native vs suffixed)	probe script already tries MackieSSD/ Mackie/Mackie 1; check ls /Volumes; mount via Finder
docker compose up: "no configuration file"	wrong directory	cd /Volumes/Mackie/ AI-Red-Team-Lab/ docker first
docker exec -it kali → "No such container"	old names were docker-kali-1 (project prefix) or container removed	docker compose exec kali bash always works; container_name: kali now pinned in compose
curl empty inside kali to localhost:3000	localhost trap — that's the container's own localhost	use webui:8080 / host.orb.internal:11434 from inside; localhost from the Mac only
Model list empty ({"models":[]})	Ollama pointed at wrong/unwritable models path	check models: line from lab-up; tail /tmp/ ollama-lab.log
*:11434 in lsof (the old leak)	Ollama.app was launched	pkill -f "/Applications/ Ollama.app" → lab-down → lab-up
WebUI chat errors	Ollama not running	lab-check → lab-up
Apt/IPv6 Network is unreachable	VM/container has no global IPv6 — instant fail, then v4 fallback	harmless; Acquire::ForceIPv4 optional

"Internet broken" in container	tested with ping — carrier drops ICMP	test TCP: curl -m5 -sI http://1.1.1.1

Two hard rules that came from real incidents:

1. Never paste command lines containing # comments — zsh passes them as arguments (caused a pkill rampage that killed Terminal itself).
2. Look before you kill: pgrep -fl "pattern" first, then kill <pid> by explicit ID. Blind pkill -f on a broad pattern is how you shoot your own session.

10. Postmortem — what this lab's construction taught (for the journal)

1. Environment propagation: launchctl setenv affects only future launches; supervisor apps respawn children with their stale env. Kill supervisors, not children.
2. Mount identity is a failure class: one SSD mounted under different names (Mackie vs Mackie 1 vs an OrbStack share) silently redirects every hardcoded path. Probe paths, don't assume them.
3. Functions shadow scripts: editing ~/bin/lab-up did nothing while a same-named zsh function existed. which <cmd> tells you which one is real.
4. Editing config doesn't restart containers: compose "Running" means skip. Name changes require down + up (recreate).
5. State drifts silently: the fence was correct when built and wrong two days later (app relapse). The canary commands are the countermeasure — run them every session start.
6. Ping lies on CGNAT. TCP or it didn't happen.
7. Changelogs are recon: "Fixed X" = "X existed, here's where to look, maybe they missed a spot."

11. Canary Commands (run every session, 15 seconds)

bash

```
lab-check
# ollama up + loopback bind only
curl -s -m3 http://127.0.0.1:11434/api/tags | head -c 60
# models served
curl -m3 -sI http://1.1.1.1 | head -n1
# internet (TCP — never ping)
curl -s -m3 http://localhost:3000/health
```

```
# target alive
curl -m3 "http://$(ipconfig getifaddr en0):11434/api/tags"
2>/dev/null || echo "SEALED" # personal side sealed
```

All five green = lab is exactly as designed. Any red = the troubleshooting table above.

12. Open Items

- F1 verdict (in progress — the IDOR matrix)
- Intel leads #2–#8 (queue in §8)
- Phase 2: reproduce known CVEs against 0.3.16 (pull from GHSA advisories, match version ranges)
- Phase 3: upgrade to current WebUI → hunt unknowns with the "bug DNA" learned
- Garak/PyRIT baseline against all three models (Module-0 criterion: any endpoint → findings report in 30 min)
- Optional: relocate Docker data dir to SSD (only if internal disk gets tight — its own session, with the unique-volume rename first)
- Re-enable UTM VM track for genuinely hostile-code experiments

— End of guide —

Save → commit → then back to the browser. The F1 matrix is still the only thing on the board. 🎯